

COS30018 Intelligent Systems – Lab 8: RNN, LSTM and Attention

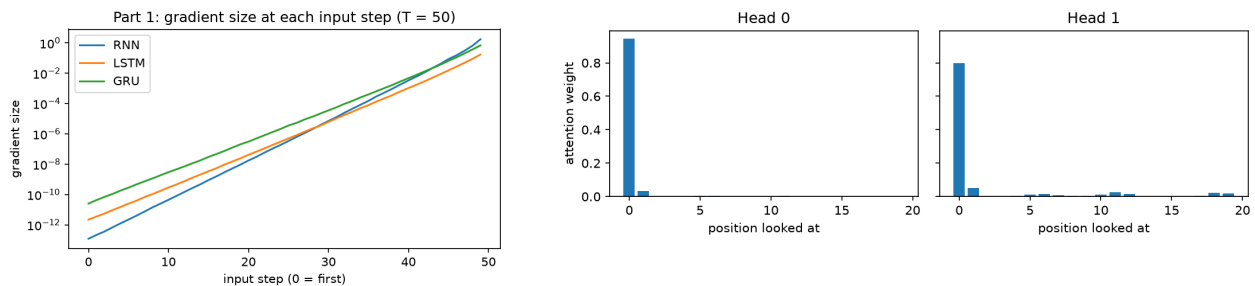
One-page summary · Anas Al Azad · Student ID: 105694136 · Notebook: Lab8_RNN_LSTM_Attention.ipynb

Results (Parts 2 and 3) – mean test accuracy over 3 seeds; task = recall the first of T random digits; chance = 0.10

Model	T = 5	T = 10	T = 20	T = 30	T = 60
RNN	1.00	0.66	0.10	–	0.10
LSTM	1.00	0.96	0.47	–	0.09
GRU	1.00	1.00	0.10	–	0.09
RNN, 128 units	–	–	0.77	–	–
LSTM, 128 units	–	–	0.59	–	–
GRU, 128 units	–	–	1.00	–	–
Self-attention	–	1.00	1.00	–	1.00
Attention, no pos. enc.	–	0.26	–	0.18	–

Experiment (32 → 128 hidden units, $T = 20$) and the ablation (no positional encoding) are the extra rows. Green = ≥ 0.90 , red = ≤ 0.20 (about chance).

Gradient plot (Part 1) and attention plot (Part 3)



Left: gradient size at each of 50 input steps (log scale) – last/first ratio RNN 1.5×10^{13} , LSTM 7.8×10^{10} , GRU 2.8×10^{10} . Right: attention of the last position at $T = 20$, averaged over 200 sequences – weight on position 0: head 0 = 0.95, head 1 = 0.80.

Checkpoint questions

1. In Part 1, which input step has the smallest gradient? What does this mean for learning the first digit?

Step 0 the very first input has the smallest gradient for all three layers: the gradient at the last step is about 1.5×10^{13} times bigger for the RNN, 7.8×10^{10} for the LSTM and 2.8×10^{10} for the GRU. But the first digit is exactly what the label is, so the weights get almost no training signal about how to store it. That's why the plain RNN drops to chance (0.10) from $T = 20$, while the gated LSTM/GRU hold on a bit longer (fine at $T = 10$).

2. In Part 2, runs with the same settings give different accuracy. What does this tell you about training RNNs on long sequences?

Only the seed changes (initial weights + data order), yet the results jump around a lot: the RNN at $T = 10$ got 0.79 / 1.00 / 0.19 and the LSTM at $T = 20$ got 0.32 / 0.18 / 0.90. Because the gradient from the early steps is so tiny, whether a run ever finds the link between step 1 and the label is basically luck. So training RNNs on long sequences is unstable you need several seeds and should look at the spread, not just one number.

3. In Part 3, accuracy drops when you remove positional encoding. Why?

Self-attention on its own doesn't know about order – it compares steps by content only, so without positional encoding the sequence is just a bag of digits and the model can't tell which one came first. The best it can do is guess the most common digit, which I calculated gives ≈ 0.27 for $T = 10$ and ≈ 0.20 for $T = 30$ – basically the 0.26 and 0.18 I got. With positional encoding every step gets its own sin/cos tag, so the last position looks straight at position 0 (weights 0.95 / 0.80) and gets 1.00.